

دانشکده مهندسی کامپیوتر

هوش مصنوعی و سیستم های خبره

ورکشاپ اول (درخت تصمیم)

دکتر عبدی

پاییز 1405

طراحان ورکشاپ : امین شیروانی ، رامین کلانتری ، دیانا سنابادی، مبینا برغمدی

پیاده‌سازی درخت تصمیم (Decision Tree)

شرح پروژه و راهنمای انجام کار

مقدمه و هدف پروژه

در این پروژه الگوریتم درخت تصمیم (Decision Tree) را از ابتدا و بدون استفاده از کتابخانه های معروف پیاده‌سازی می‌کنید و سپس با آن، دیابت داشتن یا نداشتن افراد را بر اساس داده واقعی پیش‌بینی می‌کنید. هدف فقط رسیدن به یک عدد دقت بالا نیست؛ هدف اصلی این است که با آزمایش‌های کنترل‌شده به این پرسش پاسخ بدهید:

> **پرسش اصلی پروژه**
>
> **یک درخت تصمیم باید چقدر پیچیده باشد؟**

برای پاسخ به این پرسش با مفاهیم Overfitting و Underfitting، هرس کردن (Pruning)، تفاوت معیارهای Gini و Information Gain، اعتبارسنجی متقابل (K-Fold Cross-Validation) و جست‌وجوی شبکه‌ای (Grid Search) از نزدیک کار می‌کنید.

فایل‌های پروژه

فایل	توضیح
DT_Library	کتابخانه درخت تصمیم شامل کلاس‌های Node و DecisionTree. تعدادی از متدهای آن ناقص است و شما باید آن‌ها را کامل کنید. (در نوت‌بوک با نام DT_Library ایمپورت می‌شود.)
نوت‌بوک پروژه	شامل دو بخش است: بخش اول، تست‌های واحد برای کتابخانه؛ بخش دوم، آزمایش‌ها و تحلیل روی دیتاست.

فایل	توضیح
Diabetes.csv	دیتاست خام نظرسنجی دیابت افراد. ستون هدف: Diabetes Status

قواعد کلی

- کتابخانه های هوش مصنوعی مثل scikit-learn در هیچ جای پروژه مجاز نیست. تقسیم داده را خودتان می‌نویسید و تابع K-Fold در نوت‌بوک آماده است.
- تا زمانی که همه ی تست‌های بخش اول بدون خطا اجرا نشده‌اند، سراغ بخش دوم نروید.
- داده را یک بار پیش‌پردازش و یک بار تقسیم کنید (Train/Validation/Test به نسبت 70/15/15). مجموعه Test تا پارت L دست‌نخورده می‌ماند.
- سلول‌هایی که TODO ندارند را تغییر ندهید و نام متغیرهایی را که هر سلول TODO می‌خواهد دقیقا حفظ کنید؛ سلول‌های بعدی به همین نام ها تکیه دارند.

گام اول: تکمیل فایل DT_Library

پیش از هر کار دیگری باید فایل DT_Library را کامل کنید. این فایل شامل دو کلاس Node و DecisionTree است و تعدادی از متدهای آن ناقص (TODO) است. شما باید این متدها را بر اساس تئوری الگوریتم ID3 یا CART تکمیل کنید. قبل از نوشتن کد، فرمول‌های Entropy، Gini و Information Gain و روند ساخت بازگشتی درخت را مرور کنید و روی کاغذ یک درخت کوچک بسازید.

چرا این گام مهم است؟

درستی تمام آزمایش‌های بعدی به این فایل وابسته است. اگر مثلا _find_best_split یا predict اشتباه باشد، نتایج بخش دوم بدون اطلاع شما خراب می‌شود و متوجه علت نخواهید شد. به همین دلیل بخش اول نوت‌بوک (تست‌ها) پیش از هر آزمایشی باید کامل پاس شود.

بخش اول نوت‌بوک: تست‌های واحد (Unit Tests)

هر تست یک بخش از کلاس های اصلی را می‌سنجد. سلول را اجرا کنید و اگر AssertionError دیدید، پیام خطا معمولا مشکل را مستقیم نشان می‌دهد. برخی تست‌ها از Mock (کلاس MagicMock) استفاده می‌کنند؛ یعنی یک متد کمکی با نسخه جعلی جایگزین می‌شود تا بررسی شود متد اصلی چگونه از آن استفاده می‌کند (با چه ورودی، چند بار و به چه ترتیبی). پس به امضای متدها (Signature) و ترتیب آرگومان‌ها هم دقت کنید.

تست	چه چیزی سنجیده می‌شود؟	راهنمایی
1	کلاس Node و متد is_leaf	هر گره یا «تصمیم» است یا «برگ». ویژگی‌های بی‌استفاده در هر نوع گره چه مقداری باید داشته باشند؟ فرزندان باید دقیقا همان شیء داده‌شده باشند، نه یک کپی از آن.
2	هایپرپارامترهای DecisionTree	مقادیر پیش‌فرض را از روی خود تست استخراج کنید. تغییر تنظیمات یک درخت نباید روی درخت دیگر اثر بگذارد.
3	ناخالصی وزن‌دار: _weighted_entropy و _weighted_gini	ناخالصی هر فرزند را به نسبت سهم نمونه‌هایش وزن بدهید. اگر یکی از فرزندان خالی باشد چه می‌شود؟ نتیجه نباید NaN شود.
4	_entropy، _gini و _information_gain	مقدار $0 * log_2(0)$ را چطور مدیریت می‌کنید؟ برچسب‌ها لزوماً 0 و 1 نیستند و ممکن است تعداد کلاس‌ها زیاد باشد یا یک کلاس اصلاً دیده نشود.
5	متد fit	چه چیزهایی باید هنگام آموزش روی خود شی ذخیره شود؟ آموزش دوباره روی همان شی نباید هیچ اثری از آموزش قبلی باقی بگذارد.
6	_build_tree با Mock	عمق هر فرزند نسبت به والد دقیقا چقدر تغییر می‌کند؟ هر فرزند فقط باید داده‌های شاخه خودش را بگیرد. ترتیب آرگومان‌های _is_stopping_condition را از روی تست پیدا کنید.

تست	چه چیزی سنجیده می‌شود؟	راهنمایی
7	_build_tree روی داده‌ای کوچک	این بار بدون Mock است. باید ویژگی درست، آستانه درست و برگ‌های خالص ساخته شود.
8	_find_best_split	چه زمانی هیچ شکست معتبری وجود ندارد و در این حالت چه باید برگردد؟ محدودیت min_samples_leaf را کجا اعمال می‌کنید؟ با ویژگی ثابت چه می‌کنید؟
9	_calculate_impurity	این متد فقط مسیریاب است: بر اساس criterion یکی از دو متد کمکی را صدا می‌زند. فرمول را دوباره ننویسید.
10	_calculate_information_gain	ترتیب و تعداد فراخوانی‌های متد ناخالصی (والد، چپ، راست) در تست بررسی می‌شود.
11	_predict_one	نمونه دقیقا روی آستانه به کدام سمت می‌رود؟ اگر از بازگشت استفاده می‌کنید، مراقب return باشید. _root هنگام پیش‌بینی نباید تغییر کند.
12	متدهای کمکی و آزمون سرتاسری	_get_candidate_thresholds، _split، _is_stopping_condition و اجرای کامل fit و predict. آستانه‌ها از چه مقادیری ساخته می‌شوند؟ مسئله‌ی XOR نشان می‌دهد نباید شرط توقف را به مثبت بودن gain گره بزنید. فراخوانی predict پیش از fit باید یک خطای مناسب بدهد.

راهنمایی

تست‌ها را به ترتیب شماره جلو ببرید؛ تست‌های بالاتر به متدهایی که قبلا نوشته‌اید تکیه دارند.

از نتیجه اجرای موفق هر تست یک اسکرین‌شات بگیرید و در مستند ضمیمه کنید.

بخش دوم نوت‌بوک: آزمایش‌ها و تحلیل

در این بخش از کتابخانه‌ای که در بخش اول راستی‌آزمایی کردید فقط استفاده می‌کنید: آموزش، ارزیابی و تحلیل. سلول‌هایی که با TODO مشخص شده‌اند را کامل کنید و در انتهای هر پارت به پرسش‌ها با کلمات خودتان پاسخ بدهید. نمودارها با توابع آماده رسم می‌شوند و شما فقط عددها را محاسبه می‌کنید.

پارت A : تنظیمات و توابع کمکی

در این پارت پنج تابع باید تکمیل شود. بقیه توابع (از جمله همه توابع رسم نمودار) آماده‌اند.

تابع	وظیفه	راهنمایی
accuracy_score_manual	نسبت پیش‌بینی‌های درست	ورودی‌ها می‌توانند list، آرایه یا Series باشند. اندیس pandas نباید روی مقایسه اثر بگذارد.
count_leaves	تعداد برگ‌های یک درخت آموزش‌دیده	درخت را از _root پیمایش کنید؛ هم با بازگشت و هم با یک پشته می‌شود.
post_prune	هرس بعد از ساخت درخت (Reduced-Error Pruning)	کار از پایین به بالا انجام می‌شود. فکر کنید چرا بررسی هر گره باید بعد از فراخوانی‌های بازگشتی روی فرزندانش انجام شود. خطای زیردرخت را با خطای حالتی که گره یک برگ شود (روی نمونه‌های Validation همان گره) مقایسه کنید. پس از هرس، آمار درخت (تعداد گره، عمق، اهمیت ویژگی‌ها) باید به‌روز شود.
print_tree	چاپ درخت به شکل قوانین if/else	نمونه خروجی در توضیحات تابع آمده است. تورفتگی را بر اساس عمق بسازید و گره‌هایی را که با max_depth مخفی می‌شوند مشخص کنید.
trace_path	توضیح مسیر تصمیم یک نمونه	در هر قدم، ویژگی، مقدار نمونه، آستانه، جهت حرکت و تعداد نمونه‌های گره را چاپ کنید. در پایان کلاس پیش‌بینی‌شده را برگردانید.

پارت B : آماده‌سازی داده و Baseline

در این پارت دیتاست خام Diabetes را برای مدل‌سازی آماده می‌کنید، آن را یک بار به سه بخش Train، Validation و Test تقسیم می‌کنید و یک مدل بسیار ساده (Baseline) را به‌عنوان مرجع مقایسه می‌سازید.

B.1 بارگذاری و شناخت داده

پیش از هر تصمیمی داده را بشناسید: ابعاد، نوع هر ستون، تعداد مقادیر یکتا در هر ستون، درصد مقادیر گمشده و توزیع کلاس‌های هدف. نمودارهای این سلول به شما کمک می‌کنند.

B.2 پیش‌پردازش (Preprocessing)

در سلول خالی این بخش باید دیتاست را به دو متغیر X و Y تبدیل کنید. X یک DataFrame است که فقط ستون‌های عددی دارد، اندیس یکتا دارد (reset_index) و ستون هدف در آن نیست. Yیک Series بدون مقدار گمشده با همان اندیس X است. سلول بعدی همین شرایط را خودکار بررسی می‌کند.

تمرکز این بخش بر درک فرایندها و انتخاب روش مناسب است. هر شخص می‌تواند با توجه به تحلیل داده‌ی خود از روش‌های متفاوتی استفاده کند. به‌کار بردن همه مراحل زیر اجباری نیست، اما هر مرحله‌ای را که انجام می‌دهید یا عمداً کنار می‌گذارید **باید در مستند توجیه کنید.**

مرحله	پرسش‌های راهنما
مدیریت رکوردهای ناقص (Missing Values)	• کدام ستون‌ها مقدار گمشده دارند و چند درصد؟ آیا حذف ردیف‌ها یا ستون‌ها گزینه معقولی است، یا بخش زیادی از داده را از بین می‌برد؟ • ردیفی که برچسب (Y) ندارد چه سرنوشتی باید داشته باشد؟ • «نبودن» مقدار در یک ستون دسته‌ای ممکن است خودش اطلاعات داشته باشد. چطور می‌توان آن را به‌جای حذف، نگه داشت؟ • **مقادیر گمشده عددی را همین‌جا پر نکنید. چرا؟ (سرنخ: پارت B.4)**

مرحله	پرسش‌های راهنما
مدیریت رکوردهای تکراری (Duplicate Records)	• «تکراری» دقیقا یعنی چه؟ همه ستون‌ها یکی باشند، یا فقط شناسه؟ • اگر یک نمونه هم در Train و هم در Test باشد، دقت Test چه ویژگی‌ای پیدا می‌کند؟ پس حذف تکراری‌ها را قبل یا بعد از تقسیم انجام می‌دهید؟ • ستون‌هایی که در هر ردیف مقدار متفاوتی دارند (مثل شناسه) چه اطلاعاتی برای پیش‌بینی دارند؟
تشخیص و حذف داده‌های پرت (Outlier Detection)	• با روش‌هایی مثل IQR یا Z-score آشنا شوید و روی ستون‌های عددی بررسی کنید. • هر مقدار دور از بقیه یک خطا نیست. بازه مجاز ستون‌های امتیازدهی چیست و مقدار خارج از آن بازه چه معنایی دارد؟ • درخت تصمیم برای انتخاب آستانه به مقدار دقیق یک داده حساس است یا به ترتیب مقادیر؟ با این حساب، حذف یا اصلاح شدید پرت‌ها برای این الگوریتم چقدر می‌ارزد؟ • اگر حدود تشخیص پرت را از آمار داده می‌سازید، این آمار را از کدام بخش داده باید گرفت؟
کدگذاری ویژگی‌های متنی (Encoding Categorical Data)	• درخت شما شرط عددی `x < threshold` را بررسی می‌کند؛ پس ستون‌های متنی چه سرنوشتی دارند؟ • تفاوت One-Hot و کدگذاری عددی (Label) چیست؟ برای ستونی که ترتیب دارد (مثل سطح یا کلاس) و ستونی که ترتیب ندارد (مثل نام شهر) کدام مناسب‌تر است؟ • یک ستون با تعداد زیادی دسته را اگر One-Hot کنید، تعداد ستون‌ها و زمان اجرای درخت چه می‌شود؟ چه جایگزینی دارید؟ • از ابزارهای pandas استفاده کنید؛ scikit-learn مجاز نیست.

مرحله	پرسش‌های راهنما
دسته‌بندی ویژگی‌های عددی گسترده (Binning)	• برای ستونی با صدها مقدار یکتا، درخت در هر گره چند آستانه کاندیدا را باید امتحان کند؟ این موضوع روی زمان Grid Search چه اثری دارد؟ • Binning چه چیزی را فدا می‌کند و چه چیزی به دست می‌دهد؟ اصلا به چند دسته تقسیم کنیم با چه معیاری؟ • تفاوت بازه‌بندی با عرض برابر و بازه‌بندی بر اساس چندک (Quantile) چیست؟
استانداردسازی ویژگی‌ها (Feature Scaling)	• آیا نتیجه الگوریتم شما به مقیاس ویژگی‌ها وابسته است؟ استدلال کنید و تصمیم خود را (حتی اگر «انجام ندادن» باشد) در مستند بنویسید. • اگر انجامش می‌دهید، پارامترهای آن را از کجا باید محاسبه کرد؟
انتخاب ویژگی‌های مؤثر (Feature Selection) و اهمیت ویژگی‌ها (Feature Importance)	• با ماتریس همبستگی (Correlation Matrix) و Heatmap بررسی کنید کدام ویژگی‌ها تقریبا تکرار یکدیگرند یا ارتباط چندانی با هدف ندارند. • هر ویژگی را که حذف می‌کنید باید دلیل داشته باشد. دلیلتان را بنویسید. • پس از آموزش درخت (پارت‌های I و L)، با تابع آماده feature_importance اهمیت ویژگی‌ها را ببینید و با تحلیل همبستگی خودتان مقایسه کنید. آیا همان ویژگی‌هایی که انتظار داشتید مهم‌اند؟ • (اختیاری) ویژگی‌های کم‌اهمیت را کنار بگذارید و دوباره آموزش بدهید؛ دقت چه تغییری می‌کند؟

راهنمایی

پیش از و بعد از هر مرحله پیش‌پردازش، ابعاد داده و تعداد مقادیر گمشده را چاپ کنید. این خروجی‌ها هم برای خودتان راهنماست و هم برای اسکرین‌شات مستند.

ترتیب مراحل مهم است. مثلاً حذف ردیف‌ها چه اثری روی آماری دارد که بعداً می‌سازید؟

B.3

تقسیم Train / Validation / Test

تقسیم را بدون scikit-learn و با نوشتن تابع stratified_train_test_split انجام می‌دهید. این تابع داده را به دو بخش تقسیم می‌کند، پس دوبار صدایش می‌زنیم: ابتدا 70 درصد Train و 30 درصد باقی‌مانده، سپس باقی‌مانده را نصف می‌کنیم تا Validation و Test هرکدام 15 درصد شوند.

راهنمایی

- «Stratified» یعنی نسبت کلاس‌ها در هر بخش تقریباً برابر نسبت کلاس‌ها در کل داده باشد. اگر کلاسی کم‌نمونه باشد و تصادفی تقسیم کنید، چه خطری وجود دارد؟
- ایده الگوریتم: هر کلاس را جداگانه بر بزنید و به همان نسبت از آن کلاس به هر بخش بدهید.
- از RandomState با مقدار ثابت استفاده کنید تا نتایج تکرارپذیر باشند، و اندیس اصلی ردیف‌ها را حفظ کنید.
- سلول بعدی بررسی می‌کند که هیچ نمونه‌ای گم یا تکراری نشده و بخش‌ها هم‌پوشانی ندارند.

B.4 پر کردن مقادیر گمشده عددی

مقادیر گمشده باقی‌مانده در X_train، X_val و X_test را پر کنید، با این شرط که مقدار جایگزین فقط از X_train محاسبه شود و سپس روی هر سه بخش اعمال شود.

راهنمایی

- اگر آمار را از کل داده بگیرید، اطلاعات Validation و Test چگونه به آموزش نشت می‌کند؟ به این پدیده چه می‌گویند (Data Leakage)؟
- میانگین بهتر است یا میانه؟ اگر ستونی چولگی یا مقادیر پرت داشته باشد چه؟

B.5 مدل مرجع (Baseline)

این سلول آماده است و همیشه کلاس پرتکرار Train را پیش‌بینی می‌کند. هر مدل بعدی را باید با این عدد مقایسه

کنید: درختی که دقتش به Baseline نزدیک است چه ارزشی دارد؟ به پرسش‌های انتهای پارت B (چه پیش‌پردازشی و چرا، دلیل جدا نگه داشتن Test از ابتدا، و معنای Baseline) در نوت‌بوک پاسخ بدهید.

پارت C : آزمایش اول، درختی بدون محدودیت

پرسش: اگر هیچ محدودیتی روی رشد درخت نگذاریم چه اتفاقی می‌افتد؟ فقط یک بار fit انجام می‌دهید.

- با run_experiment و پیکربندی پیش‌فرض (make_config()) یک درخت بسازید و نتیجه را در متغیر unrestricted نگه دارید.
- دقت Train، دقت Validation، دقت Baseline و پیچیدگی درخت (عمق، تعداد گره، تعداد برگ) را چاپ کنید.

> **راهنمایی**
>
> فاصله دقت Train و Validation چه چیزی را نشان می‌دهد؟ چرا یک درخت رشدکرده به‌راحتی به دقت نزدیک 100 درصد روی Train می‌رسد؟ آیا اندازه درخت با حجم داده متناسب به نظر می‌رسد؟

پارت D : آزمایش دوم، Underfitting در برابر Overfitting

پرسش: آیا درخت عمیق‌تر همیشه بهتر تعمیم می‌دهد؟ فقط max_depth را تغییر می‌دهید و بقیه تنظیمات ثابت می‌مانند.

- برای هر مقدار در depth_values یک آزمایش اجرا کنید و فهرست نتایج را در depth_results و جدول آن را در depth_df (با to_df) نگه دارید.

راهنمایی

روی نمودار مشخص کنید کدام عمق‌ها Underfitting دارند (هر دو دقت پایین) و کدام Overfitting (Train بالا و Validation پایین‌تر). آیا عمیق‌ترین درخت بهترین Validation را دارد؟ آیا می‌شود فقط با دقت Train بهترین مدل را انتخاب کرد؟

پارت E : آزمایش سوم، هرس پیش از ساخت (Pre-Pruning)

به‌جای محدود کردن عمق، دو تنظیم دیگر را بررسی می‌کنید (با max_depth=None): min_samples_split (حداقل نمونه برای شکستن یک گره) و min_samples_leaf (حداقل نمونه‌ای که هر فرزند پس از شکست باید داشته باشد).

- برای مقادیر min_split_values نتایج را در minsplit_results و minsplit_df ذخیره کنید.
- برای مقادیر min_leaf_values نتایج را در minleaf_results و minleaf_df ذخیره کنید.
- در میان همه اجراهای این پارت، کوچک‌ترین درختی (کمترین تعداد گره) را که دقت Validation آن حداکثر 0.01 با بهترین دقت فاصله دارد انتخاب و در pre_pruned ذخیره کنید؛ سپس پیکربندی و معیارهایش را چاپ کنید.

راهنمایی

با بزرگ‌تر شدن min_samples_split اندازه درخت و فاصله Train تا Validation چه می‌شوند؟ هرس پیش از ساخت در کدام لحظه ساخت درخت اثر می‌گذارد؟ این دو تنظیم چه تفاوتی دارند و کدام در نتایج شما قوی‌تر عمل کرد؟

پارت F : آزمایش چهارم، هرس پس از ساخت (Post-Pruning)

پرسش: اگر درخت از قبل بزرگ شده باشد، می‌شود بعداً ساده آن کرد؟ مسیر کار: Train سپس درخت بزرگ سپس هرس با Validation و در نهایت درخت کوچک‌تر.

- یک درخت جدید بدون محدودیت روی Train بسازید و وضعیت آن را قبل از هرس با snapshot ثبت کنید (before).
- درخت را با post_prune و مجموعه Validation هرس کنید و وضعیت بعد را ثبت کنید (after). تعداد گره‌های حذف‌شده را چاپ کنید.
- نتایج نهایی را در دیکشنری post_pruned با کلیدهای خواسته‌شده ذخیره کنید.

راهنمایی

از آنجا که Validation در تصمیم هرس هم نقش داشته، دقت Validation بعد از هرس کمی خوش‌بینانه است. چه مجموعه‌ای تخمین صادقانه‌تری می‌دهد؟ آیا درخت کوچک‌تر لزوما مدل ضعیف‌تری است؟

پارت G : مقایسه بدون هرس، هرس پیش از ساخت و هرس پس از ساخت

کد این پارت آماده است و نتایج پارت‌های C، E و F را در یک جدول و یک نمودار کنار هم می‌گذارد. اگر خطایی دیدید، نام متغیرهای unrestricted، pre_pruned و post_pruned را بررسی کنید. با توجه به نمودار فکر کنید: کدام روش ساده‌ترین درخت را داد و چقدر دقت از دست داد؟ هرس پیش از ساخت و پس از ساخت در چه لحظه‌ای عمل می‌کنند و کدام ارزان‌تر است؟ چرا از دو مدل هم‌دقت، مدل کوچک‌تر را ترجیح می‌دهیم؟

پارت H : آزمایش ششم، Gini در برابر Information Gain

همان داده و همان محدودیت‌ها (با max_depth = 4)؛ فقط criterion تغییر می‌کند. دو آزمایش بسازید و نتایج را در gain_res و gini_res نگه دارید. سلول بعدی ریشه، دقت و پیش‌بینی‌های دو درخت را مقایسه می‌کند. اگر تفاوت بزرگی ندیدید، آیا نتیجه می‌گیرید که این دو معیار همیشه معادل‌اند؟

پارت I : آزمایش هفتم، باز کردن جعبه سیاه (Explainability)

یکی از مزیت‌های بزرگ درخت تصمیم این است که می‌توان دلیل پیش‌بینی را دید.

- یک درخت جدید با max_depth = 4 روی Train بسازید و در explain_tree نگه دارید.

- اولین نمونه Validation را بردارید (sample)، برچسب واقعی، برچسب پیش‌بینی‌شده و مقادیر ویژگی‌هایش را چاپ کنید و مسیر تصمیم را با trace_path نشان دهید.

> **راهنمایی**
>
> کدام ویژگی‌ها و شرط‌ها پیش‌بینی این نمونه را تعیین کردند؟ آیا مهم‌ترین ویژگی کلی همیشه در مسیر هر نمونه‌ای ظاهر می‌شود؟ چرا؟

پارت J : آیا یک تقسیم Validation قابل اعتماد است؟ (K-Fold Cross-Validation)

عددی که از یک تقسیم Validation می‌گیرید به این بستگی دارد که کدام نمونه‌ها اتفاقاً در آن افتاده‌اند. در K-Fold، مجموعه Dev (Train و Validation) به K بخش تقسیم می‌شود و هر بار یک بخش Validation و بقیه Train است. تابع تولید Fold ها (stratified_kfold_indices) آماده است.

- تابع cross_validate(config) را بنویسید. برای هر Fold یک درخت جدید بسازید، آموزش دهید و دقت Train و Validation و تعداد گره‌ها را ثبت کنید.
- خروجی باید یک دیکشنری با کلیدهای cv_train_acc، cv_val_acc، cv_val_std، cv_nodes و fold_accs باشد (نام کلیدها را تغییر ندهید).

> **راهنمایی**
>
> چرا هر پیکربندی باید روی Fold های یکسان ارزیابی شود؟ انحراف معیار دقت Fold ها درباره اعتماد به یک تقسیم تکی چه می‌گوید؟

پارت K : تیون کردن هایپرپارامترها (Grid Search همراه با K-Fold)

در Grid Search همه ترکیب‌های یک شبکه را امتحان می‌کنید، هر کدام را با K-Fold می‌سنجید و بهترین را انتخاب می‌کنید. تعداد کل آموزش‌ها برابر است با تعداد ترکیب‌ها ضربدر K؛ پس شبکه را کوچک نگه دارید و به تخمین زمان که نوت‌بوک چاپ می‌کند توجه کنید.

- برای هر ترکیب cross_validate را اجرا و یک ردیف ذخیره کنید (ستون‌های ترکیب و همه خروجی‌ها به‌جز fold_accs). سپس grid_df را بسازید و ستون gap (اختلاف دقت Train و Validation) را اضافه کنید.
- با قاعده «ساده‌ترین مدلی که تقریبا به خوبی بهترین است» (TOLERANCE = 0.01) یک ردیف را در suggested ذخیره کنید: کمترین تعداد گره میان نامزدها و در صورت تساوی، کمترین عمق.

راهنمایی

روی Heatmap کدام ناحیه شبکه بهتر است؟ در عمق خیلی زیاد با min_samples_split کوچک چه رخ می‌دهد؟ آیا با توجه به cv_val_std پیکربندی‌های برتر واقعاً با هم متفاوت‌اند؟ چرا این روش از انتخاب بهترین نقطه روی یک تقسیم تکی معتبرتر است؟

پارت L : مدل نهایی، ارزیابی روی Test و رسم درخت

هشدار

مجموعه Test فقط در این پارت و فقط یک بار استفاده می‌شود. پیکربندی نهایی باید پیش از دیدن دقت Test انتخاب شده باشد.

- پیکربندی پیشنهادی پارت K (یا انتخاب خودتان با ذکر دلیل) را بخوانید و final_tree را روی کل مجموعه Dev آموزش دهید. پیکربندی، دقت روی Dev، برآورد CV و پیچیدگی را چاپ کنید.
- پیش‌بینی‌های Test را در test_pred و دقت آن را در test_acc نگه دارید.
- برای هر کلاس مقادیر TP، FP و FN را از روی برچسب‌های واقعی و پیش‌بینی‌شده Test بشمارید و Precision، Recall و F1 را بسازید (report_df). میانگین‌های macro و weighted را در avg_df قرار دهید. در صورت صفر شدن مخرج، مقدار 0.0 بگذارید.

راهنمایی

Recall وزن‌دار باید دقیقاً برابر با Accuracy شود؛ نوت‌بوک همین را بررسی می‌کند و اگر نشد، شمارش TP و FN را مرور کنید.

با ماتریس درهم‌ریختگی (Confusion Matrix) توضیح دهید کدام خانه‌ها TP، FP و FN یک کلاس را می‌سازند و درخت بیشتر کدام کلاس‌ها را با هم اشتباه می‌گیرد.

در پایان درخت نهایی را هم به‌صورت قوانین (print_tree) و هم به‌صورت تصویر (plot_tree) نمایش دهید.

جمع‌بندی نهایی

نوت‌بوک در انتها یک تأمل نهایی (Final Reflection) دارد که باید با کلمات خودتان و بر اساس نتایج خودتان پاسخ دهید، و یک جدول خلاصه که یا دستی یا با سلول آماده پر می‌شود. دقت Test فقط در ردیف مدل نهایی گزارش می‌شود.

آنچه باید تحویل دهید

- فایل DT_Library تکمیل‌شده که همه تست‌های بخش اول را پاس می‌کند.
- نوت‌بوک کامل‌شده و **اجراشده**، با خروجی همه سلول‌ها و پاسخ پرسش‌ها.
- مستند پروژه (Word یا LaTeX، تحویل به‌صورت PDF) مطابق ضوابط مستندسازی ابتدای همین سند: توضیح بلاک‌های کد، اسکرین‌شات از هر بخش، فونت B Nazanin اندازه 14.
- در مستند، گزارشی کامل از مسیر انجام کار و چالش‌ها، اجراهای گرفته‌شده و روند پیشرفت پروژه ارائه دهید.
- هرگونه تحلیل اضافه مفید و خلاقیت (مثلا تحلیل آماری جداگانه ویژگی‌ها) می‌تواند نمره امتیازی داشته باشد.
- همه موارد بالا را در یک فایل zip با نام `DT_StudentID.zip` در کوئرا قرار دهید.